

Руководство по декодированию потока

Специальная монография по обратной половине кодека DataShield — конвейерному декодированию FEC-потока в накопители приёма. Документ дополняет [AlgorithmGuide.ru.md](#) (разделы 5–9 дают обзор): здесь та же тема разобрана формально — постановка задачи, модель повреждений, архитектура конвейера и её инварианты, алгоритмы прямого прохода и адресной перепривязки с обоснованиями полноты, оценки стоимости и точные границы гарантий. Инженерный пересказ с совпадающей нумерацией разделов — [DecoderGuide.Engineer.ru.md](#); вводное изложение «с нуля» — [DecoderGuide.Tutorial.ru.md](#). Прямое преобразование — семейство [EncoderGuide.*](#); арбитраж результата — семейство [AssemblyGuide.*](#).

1. Постановка задачи

Вход — байтовая последовательность x произвольной структуры: подпоследовательность пакетов FEC-потока, перемежающаяся произвольным мусором; границы пакетов не маркированы. Требуется построить множество слотов приёма $\Sigma = \{\sigma_f\}$ — по одному на каждый файл f , представленный распознаваемыми пакетами, — такое, что:

- Точность.** В слот попадают только пакеты, криптографически связанные с заголовком файла (ложное принятие $\leq 2^{-72}$ на сектор, 2^{-192} на заголовки).
- Полнота.** Любой неповреждённый пакет $p \in x$, для которого выполнена проверка принадлежности (заголовок — всегда; сектор — при известном заголовке своего файла на момент доступа к p либо после перепривязки), принят хотя бы один раз.
- Отсутствие задвоений.** Число подтверждений версии пропорционально числу физических копий пакета в x , а не числу проходов декодера.

Выход сборки из слота — предмет [AssemblyGuide.Academic.ru.md](#); здесь сборка не рассматривается.

2. Модель повреждений

Канал искажает вход четырьмя способами: (а) стирание интервалов; (б) вставка мусора произвольного алфавита; (в) склейка/рассинхронизация границ; (г) подмена пакетов. Ответы конвейера: (а) — избыточность кодера (ЕСС, повторение заголовков) плюс восстановление на сборке; (б) — фильтр алфавита (txt) и побайтовый сдвиг окна; (в) — скользящее окно с шагом 1; (г) — хешевая привязка $H5$ / $D3$ и финальный арбитраж $H3$.

3. Архитектура конвейера

Конвейер — цепочка модулей над интерфейсами [IDataSource](#) / [IDataProcessor](#): источник объявляет [DataReady \(вычитка\)](#); обработчик подключается [Attach](#), обрабатывает куски в [ProcessChunk](#) под [SyncRoot](#), публикует выход через байтовый ([Emit](#)) или пакетный ([EmitPacket](#))

режим буфера и является источником для следующего звена. `Complete()` — сигнал конца входа с выдачей остатка; вызывается строго по цепочке от головы.

Инвариант J1 (неделимость пакетов). Пакетный режим вывода не разрезает пакеты границами выдачи: `DataReady` выносит целые пакеты при достижении порога `BufferSize`.

Инвариант J2 (прозрачность ошибок). При сбое источника его `Error ≠ null` и `Completion` завершена этим исключением; каждый обработчик делегирует `Error` вышестоящему. Ошибка не поглощается ни одним звеном.

Инвариант J3 (сборка пакетов из кусков). `_pending`-буфер накопителя восстанавливает пакеты из кусков с произвольными границами; потеря и дублирование байтов на стыках исключены.

4. Фильтр Base64

`ByteRangeFilter` — детерминированный отображатель `byte → {пропустить, выбросить}` по таблице 256 флагов; для txt-входа пропускаются `A-Z U a-z U 0-9 U {+, /}`. Так как пакет кодируется в ровно 100 символов без `=`, алфавит без `=` полон для легитимного входа.

Предложение 4.1 (безвредность фильтра). Фильтр не удаляет ни одного символа легитимной Base64-кодировки пакетов; удаляются только символы, не входящие в алфавит, которые в кодировке пакета не встречаются. ■

5. Скользящее окно: прямой проход

`SlidingWindowScanner(w, h)` с окном `w` (100 txt / 75 bin) и обработчиком `h: window → (advance ≥ 1, packet?)`. Прямой проход: при неуспехе продвижение 1, при успехе — `w` (или возвращённое обработчиком). Сканер удерживает весь вход `R` (retained); позиция прохода `s`; граница выдачи `f ≤ s + w`.

Предложение 5.1 (полнота покрытия начал). Пусть неповреждённый пакет `p` начинается в позиции `i` входа и его распознавание обработчиком даёт успех. Тогда прямой или повторный проход, проверяющий позиции `i, i+1, ..., i+w-1`, содержит позицию `i` как старт окна. Следствие: шаг 1 при неуспехе гарантирует, что ни одно начало пакета не пропущено. ■

Предложение 5.2 (стоимость рассинхронизации). Восстановление синхронизации после вставки `g` байт мусора требует не более `g + w - 1` неудачных окон; для txt-потока с порчей одного символа строки — до `w - 1 = 99` неудачных окон. ■

6. Распознавание пакетов

Предикат `Recognizes(p)` накопителя: `IsHeader(p) := Trunc24(SHA-256(p[0..51])) = p[51..75]` — автономен; для сектора — дизъюнкция по слотам: `num(p) ∈ [0, T_σ)` и `VerifySectorPacket(p, H5_σ)`.

Предложение 6.1 (вероятности ложного принятия). Для случайного 75-байтного кандидата: заголовок — 2^{-192} ; сектор при известном `H5` — 2^{-72} ; для потока длины `L` пакетов объединённая оценка $\approx L \cdot 2^{-72}$. ■

Предложение 6.2 (зависимость сектора от заголовка). Проверка сектора вычислима только при `H5`; из секторов `H5` не выводим (стойкость SHA-256). Следовательно, до первого заголовка файла ни один его сектор неопознаваем; проблема решается перепривязкой (раздел 10). ■

7. Накопитель: куски → пакеты → слоты

`StreamProcessor.ProcessChunk` собирает пакеты (J3) и классифицирует: заголовок — побайтовое сравнение с известными слотами (совпадение → инкремент счётчика копий; новый → `ReadFrom`, `H5`, слот, событие `HeaderAccepted` вне блокировки); сектор — `AddSector(num, копия payload)` во все слоты, прошедшие проверку. Метрики `FileCount`, `TotalReceived*`, `TotalCollisionSectorCount` — снимки под блокировкой, чтение параллельно приёму.

Инвариант J4 (потокобезопасность). Все мутации состояния под `SyncRoot` / `_state`; события внешним наблюдателям — вне блокировки.

8. Слоты приёма и версии

`ReceptionSlot` — состояние одного файла: заголовок, счётчик его копий, словарь «номер → версии со счётчиками». Инварианты сортировки и накопления — `AssemblyGuide.Academic.ru.md` (разделы 3–4). Внутри конвейера слот — пассивный приёмник; все решения принимает классификация накопителя.

9. Многофайловость

Слотов столько, сколько различных заголовков. Сектор проверяется против всех слотов; в силу предложения 6.1 пересечение (сектор, валидный для двух разных `H5`) практически невозможно.

Предложение 9.1 (независимость файлов). Приём одного файла не влияет на счётчики другого; перепривязка по заголовку `f` проверяет принадлежность только `f`. ■

10. Адресная перепривязка

При `HeaderAccepted(заголовок f)` декодер запрашивает у обоих сканеров повторный проход по удержанным данным с предикатом `RebindWindow`: декод Base64-окна (txt), номер в `[0, T_f)`, `VerifySectorPacket(·, H5_f)`. Активный сканер исполняет запрос отложено с границей `f_pos` (позиция выдачи прямого прохода на момент запроса); простаивающий — немедленно по всему пройденному объёму, с временным переподключением накопителя.

Предложение 10.1 (полнота перепривязки). Всякий сектор файла `f`, лежащий в удержанных данных до границы перепривязки и неповреждённый, будет распознан повторным проходом (по предложению 5.1, применённому к исчерпывающему перебору позиций). ■

11. Повторное сканирование

Два свойства повторного прохода. **Исчерпываемость:** окно проверяется на каждой позиции региона `[0, bound)` — без прыжков; тем самым закрываются пропуски прямого прохода, где прыжок на `w` после успеха мог перескочить начало перекрывающегося пакета. **Граница** `f` фиксируется в `OnDelivering` до вызова обработчиков `DataReady`; повторный проход области за `f` запрещён.

Предложение 11.1 (отсутствие задвоения). Байт входа участвует в выдаче повторного прохода с данным предикатом не более одного раза: граница монотонна по выдачам прямого прохода и не расширяется повторными проходами. Следствие: счётчик подтверждений версии не превосходит числа физических копий пакета в входе. ■

Оценка стоимости. Повторный проход — $O(\text{bound} \cdot w)$ операций окна; память сканера — $O(|X|)$.

12. Финализация конвейера

`Complete` вызывается по цепочке: фильтр → сканер → накопитель. Финальная выдача сканера поднимает `f` и исполняет отложенные перепривязки.

Предложение 12.1 (полнота финализации). После `processor.Complete()` все данные, выданные прямым проходом, охвачены перепривязками по всем принятым к этому моменту заголовкам. Пропуск финализации нарушает свойство 2 постановки задачи. ■

13. Прогресс и отмена

Прогресс — `потреблено × 100 / |X|` по событию `ConsumedAdvanced`; фаза `HeaderSearch` до первого слота, далее `SectorSearch`; терминальный отчёт `Done` (100). Отмена: `CancellationToken` проверяется в отчёте прогресса и в ожидании `Completion`; исключение наружу, накопленное состояние сохраняется (J4).

14. Сборка

`TryAssemble(header)` — сериализация, побайтовый поиск слота, делегирование `slot.TryAssemble(rs, ...)`. Свойства результата — `AssemblyGuide.Academic.ru.md`.

15. IO-модуль и канал ошибок

Источники `ByteArraySource` / `StreamSource` / `FileSource` наследуют `BufferedSourceBase`:
исключение чтения фиксируется в `Error`, источник останавливается, `Completion` фолтится (J2).
Приёмники `StreamDataWriter` / `FileDataWriter` / `ByteListWriter` / `PreallocatedBufferWriter`
записывают результат сборки.

16. Сводка гарантий и пределов

Инварианты: **J1** неделимость пакетов; **J2** прозрачность ошибок; **J3** сборка из кусков; **J4** потокобезопасность.

Гарантии:

1. Точность опознания: $2^{-192} / 2^{-72}$ (6.1); подмена требует прообраза усечённого SHA-256.
2. Полнота приёма неповреждённых пакетов при шаге 1 и исчерпывающем повторном проходе (5.1, 10.1, 12.1).
3. Отсутствие задвоения подтверждений (11.1).
4. Независимость многофайлового приёма (9.1).
5. Доставка ошибок ввода-вывода вызывающему коду (J2).

Пределы: память `O(|X|)` на сканер (плата перепривязки); повторный проход `O(bound · w)`; перепривязка покрывает только удержанные данные; секторы за пределами границы выдачи прямого прохода добираются только последующими выдачами/финализацией.

Числовая сводка: окно 100/75; шаг неудачи 1; вероятности $2^{-192}/2^{-72}$; стоимость порчи символа ≤ 99 окон; порог выдачи буферов 1200–4096 байт.